



Consultas Avanzadas y Relaciones

Clase 6 - Consultas anidadas en SQL



¿Qué aprenderás en esta sesión?

Aprenderemos a crear consultas anidadas (subqueries) para la obtención y modificación de datos en SQL, reconociendo su sintaxis, reglas de implementación y los contextos más adecuados para su aplicación.

Comprenderemos en profundidad cómo se evalúan, cuándo usarlas y cómo integrarlas en tareas reales del entorno productivo, como reportes automatizados, indicadores comparativos, y filtrado de información contextual.

Desafío inicial

El problema

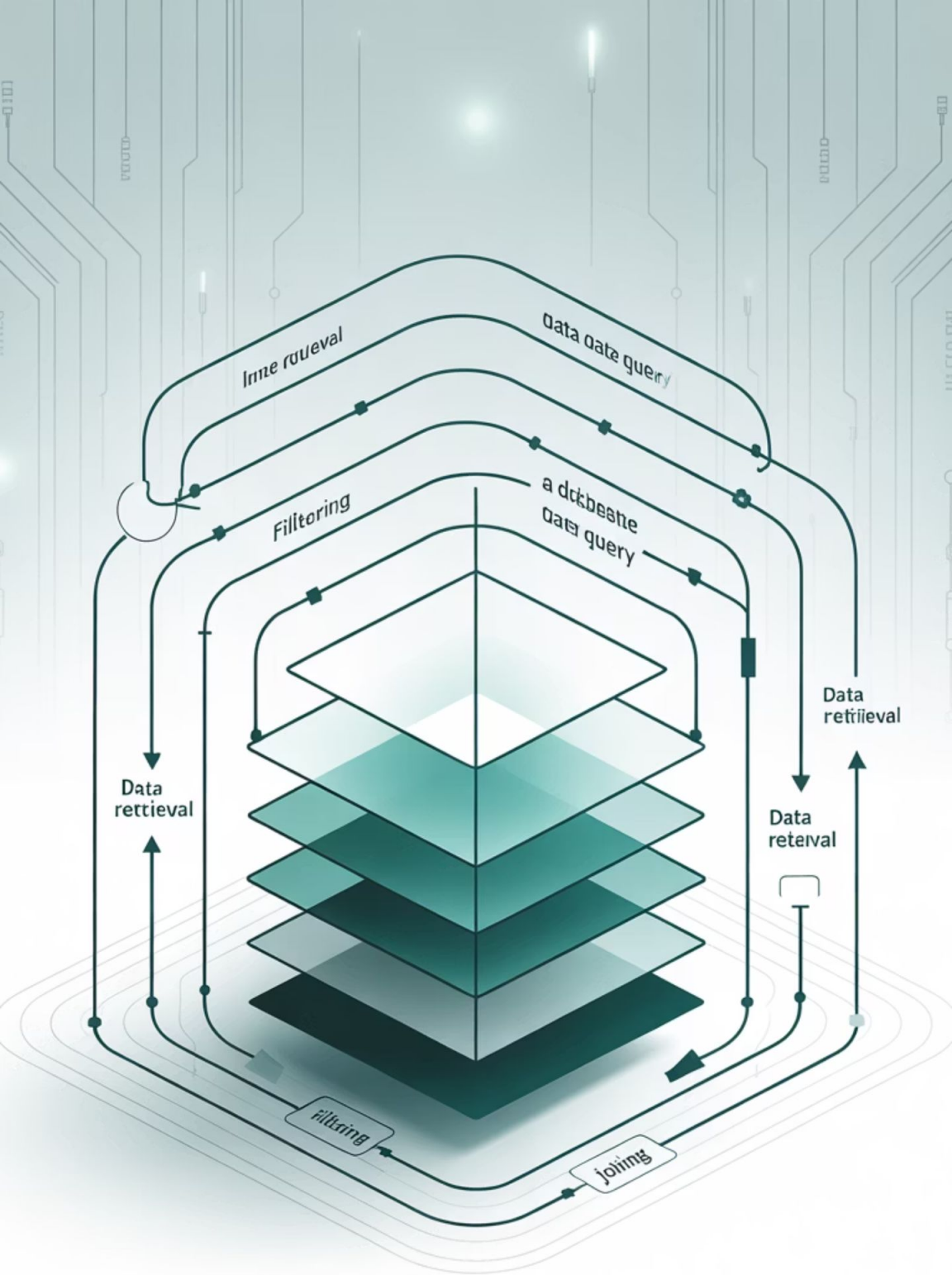
Eres parte del equipo de inteligencia de negocios de una cadena de supermercados con cobertura nacional. Se te ha solicitado elaborar un informe que muestre los productos cuyo precio de venta está por encima del promedio de su categoría, pero solo si dicha categoría tiene un volumen de ventas menor al promedio general. Esta información será utilizada por el área de compras para renegociar contratos con proveedores.

¿Cómo podemos combinar criterios agregados en distintas tablas para filtrar productos específicos, sin generar múltiples consultas intermedias ni crear tablas auxiliares?

La solución

Usar subconsultas anidadas para obtener de forma dinámica los valores promedio por categoría y de ventas globales, integrándolos como parte de una única instrucción SQL que permita extraer información precisa, optimizando recursos computacionales y garantizando trazabilidad lógica.





Proceso de evaluación de subqueries

¿Qué es una subconsulta y cómo se evalúa?

En escenarios productivos reales, especialmente en áreas de analítica y control de gestión, se requiere extraer información compleja desde bases de datos sin crear estructuras temporales ni alterar los esquemas existentes. Las subconsultas permiten obtener respuestas precisas de forma encapsulada y reutilizable.

Una subconsulta es una sentencia SELECT completa embebida dentro de otra instrucción SQL. Estas estructuras son evaluadas primero, generando un resultado temporal que luego es utilizado por la consulta principal. Pueden ubicarse en las cláusulas SELECT, FROM, WHERE y HAVING.

Subtipos de subqueries

1

Escalares

Devuelven un único valor (ej. una media).

2

De columna

Devuelven una lista de valores (una sola columna).

3

De tabla

Devuelven varias filas y columnas (requieren alias en FROM).

Ejemplo 1: Subconsulta escalar

```
SELECT nombre FROM productos
WHERE precio > (
    SELECT AVG(precio)
    FROM productos
);
```

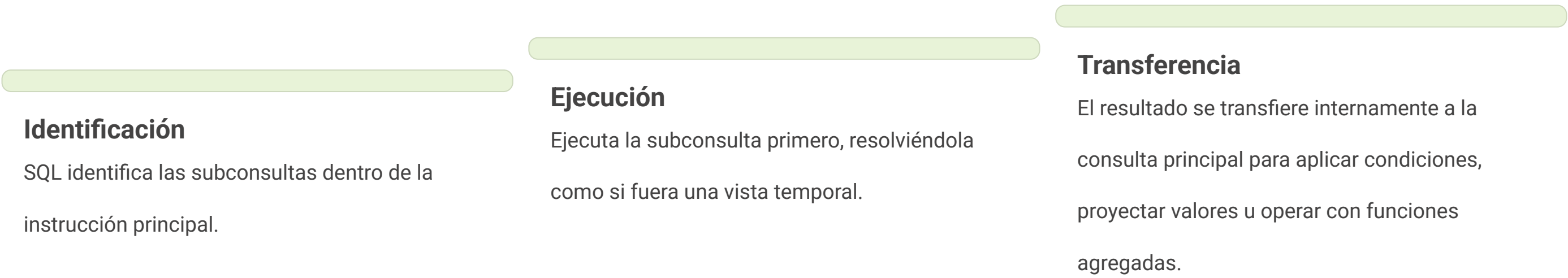
En este caso, el objetivo es identificar productos de alto precio con respecto al promedio general de todos los productos. Puede usarse para decisiones de pricing o alertas de anomalías.

Ejemplo 2: Subconsulta de columna

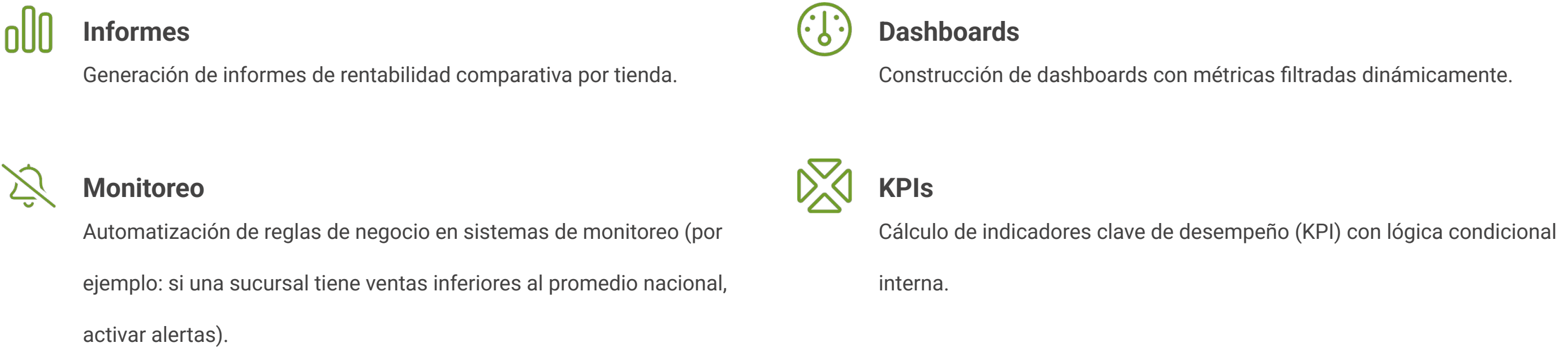
```
SELECT nombre FROM productos
WHERE categoria_id IN (
    SELECT id
    FROM categorias
    WHERE tipo = 'Importado'
);
```

Filtra productos pertenecientes a categorías específicas, como aquellas asociadas a bienes importados. Es común en auditorías de costos logísticos o en evaluaciones de políticas arancelarias.

Flujo de evaluación



Aplicación productiva



Buenas prácticas y errores comunes

Buenas prácticas

- Validar subconsultas individualmente antes de integrarlas.
- Limitar subconsultas a operaciones lógicas claras y justificadas.
- Optimizar consultas utilizando índices y evitando funciones no deterministas dentro de subqueries.
- Comentar el propósito de la subconsulta cuando sea parte de sistemas colaborativos.

Errores comunes

- Usar subconsultas no correlacionadas donde se requiere dependencia del contexto.
- Confundir subconsultas escalares con subconsultas de conjunto, lo que puede generar errores de tipo o cardinalidad.
- Anidar múltiples subconsultas sin revisar su impacto en la performance.
- Omitir alias cuando la subconsulta está en la cláusula FROM, generando errores de compilación.

Comprender qué es y cómo se ejecuta una subconsulta nos permite modelar situaciones complejas de negocio en una única instrucción SQL, aumentando la precisión y eficiencia de nuestros análisis. Para que estas estructuras funcionen correctamente, debemos conocer las reglas que regulan su implementación. Pasemos ahora a revisar en detalle dichas reglas.

Reglas asociadas a la implementación de subqueries

Ubicación y sintaxis permitida

Las subconsultas pueden implementarse de distintas maneras dependiendo de la lógica del negocio y de la información que se desea recuperar. La correcta elección del tipo y ubicación de una subconsulta impacta directamente en la claridad, funcionalidad y rendimiento de la consulta.

Ubicaciones válidas

WHERE Establecer condiciones con valores comparativos.	Usar subconsultas como tablas temporales (derivadas). FROM
SELECT Proyectar valores individuales o calculados.	HAVING Aplicar condiciones sobre agregados post agrupamiento.

Ejemplo en FROM (tabla derivada)

```
SELECT promedio
FROM
(
    SELECT AVG(precio) AS promedio
    FROM productos
)
AS tabla_temporal;
```

Ejemplo en SELECT

```
SELECT
    nombre,
    (SELECT AVG(precio) FROM productos) AS promedio_general
FROM productos;
```

Reglas técnicas para la implementación

Alias obligatorio

En subconsultas dentro de FROM, se debe usar un alias para que la consulta principal pueda referirse a sus columnas.

Compatibilidad de tipos de datos

El resultado de la subconsulta debe coincidir con el tipo de dato esperado por la consulta externa. Por ejemplo, no se puede usar una subconsulta que retorna texto donde se espera un número.

Uso de operadores correctos

- = para subqueries escalares.
- IN para listas de valores.
- EXISTS para comprobar existencia de filas.
- ALL / ANY para comparaciones condicionales sobre subconjuntos.

Errores comunes y buenas prácticas

Errores comunes

- Omitir alias de tabla derivada.
- Utilizar = con subconsultas que devuelven múltiples resultados.
- No verificar tipos de datos equivalentes.
- Confundir IN con EXISTS sin considerar diferencias de comportamiento.

Buenas prácticas

- Usar paréntesis claros y sangrías que identifiquen la jerarquía de las subconsultas.
- Mantener subconsultas lo más simples posible.
- Aplicar LIMIT cuando no se requiere todo el conjunto.
- Asegurar que las subconsultas devuelvan datos válidos aún si las tablas están vacías.

Manejar correctamente la ubicación y sintaxis de subqueries nos garantiza una construcción lógica precisa. No obstante, debemos ir más allá: es fundamental saber cuándo conviene usar una subconsulta en lugar de otras herramientas como los JOINS. A continuación, exploraremos los criterios clave para decidir cuándo utilizar subconsultas en el trabajo profesional.

Cuándo es recomendable utilizar subqueries

En ambientes corporativos donde se requiere responder con agilidad a requerimientos analíticos, el diseño eficiente de consultas es clave. Elegir entre usar subqueries o JOINS depende de múltiples factores, incluyendo legibilidad, rendimiento y reusabilidad del código.

Subqueries son preferibles cuando



Se requiere encapsular una lógica sin alterar la estructura principal.



Se necesitan cálculos agregados dependientes del contexto (como promedios por categoría).



Se desea aplicar filtros que dependen del resultado de otra condición evaluada previamente.



Se trabaja con herramientas que no permiten múltiples joins (ej. ciertos editores visuales).

Comparación estructurada

Aspecto	Subquery	JOIN
Legibilidad	Alta (cuando se encapsula lógica compleja)	Menor en estructuras con múltiples relaciones
Rendimiento	Adecuado en consultas simples	Superior en grandes volúmenes
Modularidad	Alta (puede reutilizarse y separarse)	Baja (acoplamiento fuerte entre tablas)
Control de contexto	Más flexible	Más rígido (todo debe estar relacionado)

Tabla 1. Comparación estructurada

Fuente: Desafío Latam

Ejemplo práctico:

Obtener empleados con salario mayor al promedio de su departamento:

```
SELECT nombre
FROM empleados e
WHERE salario > (
    SELECT AVG(salario)
    FROM empleados
    WHERE departamento_id = e.departamento_id
);
```

Este patrón es ampliamente utilizado en sistemas de evaluación de desempeño, ranking salarial, y segmentación de recursos humanos.

Errores comunes y buenas prácticas

Errores comunes

- Implementar subconsultas anidadas excesivamente profundas, afectando performance y mantenibilidad.
- Usar subqueries donde un JOIN simple resolvería mejor el problema.

Buenas prácticas

- Usar subqueries cuando se requiera encapsulación lógica o cálculo de agregados.
- Comparar la eficiencia entre JOIN y subquery usando EXPLAIN PLAN u otros métodos de diagnóstico del motor de base de datos.
- Documentar cuándo una subconsulta fue elegida por razones técnicas o de claridad.

Ya comprendemos el rol estratégico de las subqueries en la arquitectura de soluciones SQL. Su uso adecuado puede marcar la diferencia entre una consulta funcional y una consulta optimizada, comprensible y escalable.

Actividad guiada:

**Subconsultas aplicadas al
análisis de ventas y
márgenes**



Objetivo de la actividad

Aplicar subconsultas escalares, de columna y correlacionadas para resolver un problema real vinculado a la gestión de precios y rentabilidad en una base de datos relacional, integrando filtros y cálculos agregados en una única instrucción SQL.

Situación problema

Trabajas como analista de datos en una empresa de retail. Se te encarga identificar los productos cuyo precio de venta es superior al promedio de su categoría y cuya categoría tiene un margen de ganancia promedio inferior al 20 %. Esta información es clave para que el área de compras pueda renegociar condiciones con proveedores o ajustar el mix de productos.

Además, se requiere que el resultado incluya:

- Nombre del producto
- Nombre de la categoría
- Precio de venta del producto
- Margen promedio de la categoría



Instrucciones

Crear las tablas base con los siguientes comandos:

```
CREATE TABLE categorias (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50)  
);  
  
CREATE TABLE productos (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  categoria_id INT,  
  precio_venta DECIMAL(10,2),  
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)  
);  
  
CREATE TABLE margen_categoria (  
  categoria_id INT PRIMARY KEY,  
  margen_promedio DECIMAL(4,2),  
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)  
);
```



Insertar datos ficticios

```
INSERT INTO categorias VALUES
```

```
(1, 'Lácteos'),  
(2, 'Carnes'),  
(3, 'Panadería');
```

```
INSERT INTO productos VALUES
```

```
(1, 'Leche Entera', 1, 1.20),  
(2, 'Queso Fresco', 1, 3.50),  
(3, 'Yogurt Natural', 1, 2.10),  
(4, 'Carne Molida', 2, 5.60),  
(5, 'Pollo Entero', 2, 4.80),  
(6, 'Pan Baguette', 3, 1.50);
```

```
INSERT INTO margen_categoria VALUES
```

```
(1, 0.15),  
(2, 0.22),  
(3, 0.18);
```

Ejecutar la siguiente consulta:

```
SELECT
```

```
p.nombre AS producto,  
c.nombre AS categoria,  
p.precio_venta,  
m.margen_promedio
```

```
FROM productos p
```

```
JOIN categorias c ON p.categoria_id = c.id
```

```
JOIN margen_categoria m ON c.id = m.categoria_id
```

```
WHERE p.precio_venta > (
```

```
    SELECT AVG(precio_venta)
```

```
    FROM productos p2
```

```
    WHERE p2.categoria_id = p.categoria_id
```

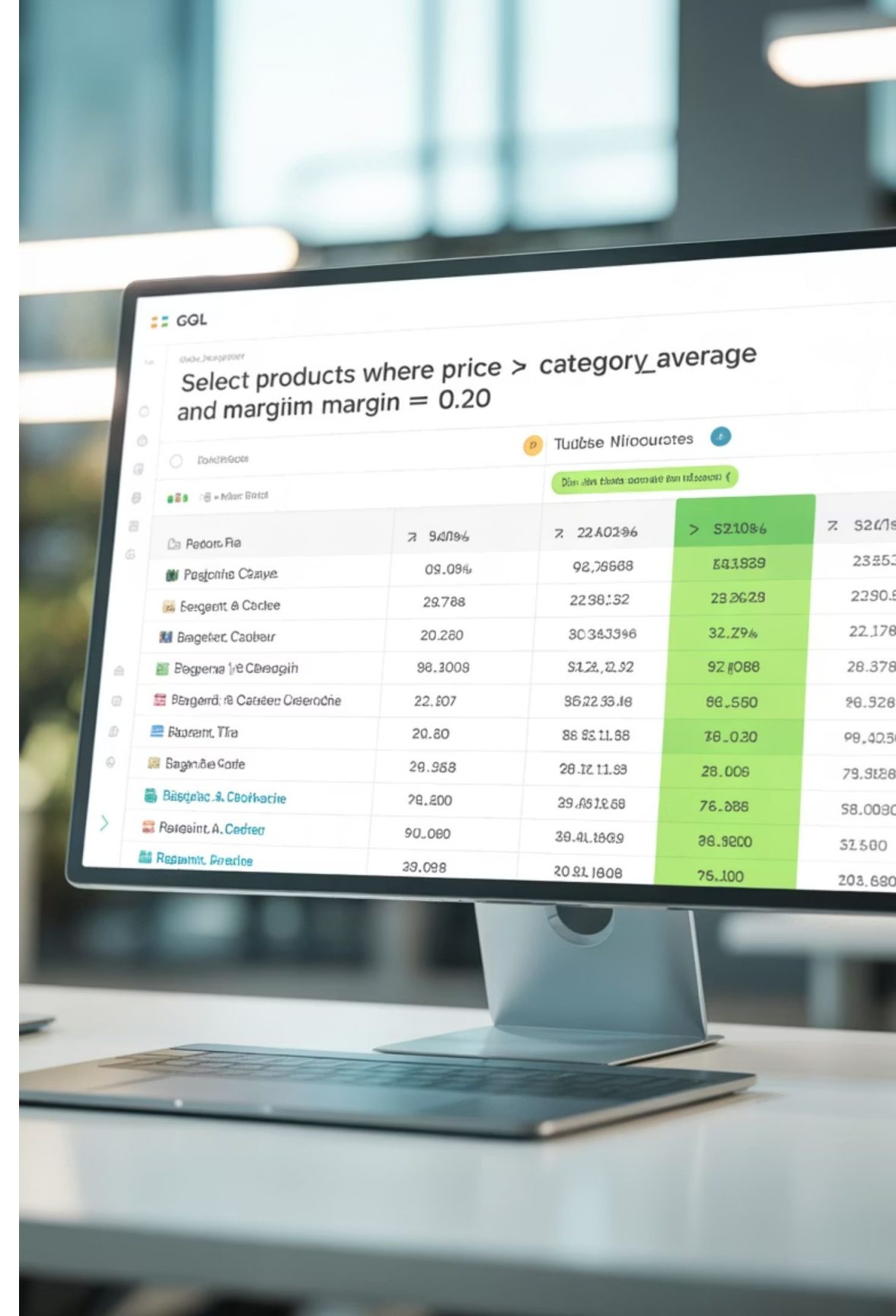
```
)
```

```
AND m.margen_promedio < 0.20;
```

Analizar el resultado: ¿Qué productos cumplen ambas condiciones?

Materiales necesarios

- Plataforma de ejecución SQL (pgAdmin, MySQL Workbench, DBeaver, DB-Fiddle o similar).
- Proyector/pizarra para guiar en tiempo real.
- Plantilla de reporte para registrar decisiones tomadas (opcional).



Actividad práctica autónoma:

**Detección de productos
estratégicos por precio y
rotación**



Objetivo de la actividad

Aplicar subconsultas escalares y correlacionadas para detectar productos que, por su comportamiento en precio y rotación, deben ser priorizados en el próximo ciclo de abastecimiento.

Contexto

Formas parte del equipo de abastecimiento de una empresa mayorista que opera con múltiples líneas de productos. Tu jefatura te solicita identificar los productos con precio por encima del promedio de su línea, pero cuya rotación mensual está por debajo del promedio general de la empresa. Este análisis permitirá optimizar las compras para evitar sobrestock de productos con baja rotación y alto costo.

Los datos se encuentran en las siguientes tablas:

- `lineas(id, nombre)`
- `productos(id, nombre, linea_id, precio_unitario)`
- `rotacion(id_producto, rotacion_mensual)` — expresada en unidades vendidas al mes



Instrucciones

Crear las tablas necesarias

```
CREATE TABLE lineas (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50)  
);  
  
CREATE TABLE productos (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(100),  
  linea_id INT,  
  precio_unitario DECIMAL(10,2),  
  FOREIGN KEY (linea_id) REFERENCES lineas(id)  
);  
  
CREATE TABLE rotacion (  
  id_producto INT PRIMARY KEY,  
  rotacion_mensual INT,  
  FOREIGN KEY (id_producto) REFERENCES productos(id)  
);
```



Insertar datos ficticios

```
INSERT INTO lineas VALUES
```

```
(1, 'Aseo'),  
(2, 'Bebestibles'),  
(3, 'Alimentos Secos');
```

```
INSERT INTO productos VALUES
```

```
(1, 'Detergente Líquido', 1, 4.50),  
(2, 'Desinfectante', 1, 3.90),  
(3, 'Jugo Natural 1L', 2, 2.10),  
(4, 'Agua Mineral 1.5L', 2, 1.20),  
(5, 'Arroz 1Kg', 3, 2.50),  
(6, 'Lentejas 1Kg', 3, 2.70);
```

```
INSERT INTO rotacion VALUES
```

```
(1, 90),  
(2, 60),  
(3, 150),  
(4, 300),  
(5, 200),  
(6, 100);
```



Escribe y ejecuta una consulta que:

- Compare el precio del producto con el promedio de su línea.
- Compare su rotación con el promedio de rotación de todos los productos.
- Devuelva nombre del producto, nombre de la línea, precio y rotación.

```
SELECT
  p.nombre AS producto,
  l.nombre AS linea,
  p.precio_unitario,
  r.rotacion_mensual
FROM productos p
JOIN lineas l ON p.linea_id = l.id
JOIN rotacion r ON p.id = r.id_producto
WHERE p.precio_unitario > (
  SELECT AVG(precio_unitario)
  FROM productos p2
  WHERE p2.linea_id = p.linea_id
)
AND r.rotacion_mensual < (
  SELECT AVG(rotacion_mensual)
  FROM rotacion
);
```

Análisis de resultados

- 1. Analiza los resultados: ¿Cuáles productos representan un riesgo de sobrestock?, ¿Qué decisiones podrías recomendar con esta información?
- 2. Documenta tus decisiones en la siguiente tabla:

Producto	¿Sobreprecio?	¿Baja rotación?	¿Recomienda abastecer?	Justificación breve

Materiales necesarios

- Script SQL.
- Editor SQL local o en línea.
- Hoja de cálculo o plantilla para registrar análisis (opcional).

Resumen



Comprendimos qué es una subconsulta y cómo se evalúa dentro de una instrucción SQL más amplia.



Identificamos los distintos tipos de subqueries: escalares, de columna, de tabla y correlacionadas.



Conocimos las reglas sintácticas y estructurales clave para su correcta implementación.



Analizamos cuándo conviene utilizar subconsultas en lugar de otras herramientas como los JOIN.



Aplicamos subconsultas en escenarios reales relacionados con gestión de precios, rotación, márgenes y categorías de productos.



Próxima sesión...

En la siguiente sesión comenzaremos la Unidad de Manipulación y Gestión de Datos, abordando la Creación y manipulación de tablas (Parte I).

Aprenderemos a construir estructuras relacionales desde cero mediante la instrucción `CREATE TABLE`, definiendo tipos de datos adecuados para cada columna, y aplicando restricciones como `PRIMARY KEY`, `NOT NULL`, `UNIQUE` y `CHECK`.

Este conocimiento es esencial para garantizar la calidad del diseño de bases de datos y facilitar futuras consultas analíticas o de modificación.

Preguntas de cierre

¿Cómo se evalúa una subconsulta correlacionada y en qué se diferencia de una subconsulta no correlacionada?

¿Qué riesgos representa utilizar subconsultas en cláusulas WHERE que retornan múltiples valores inesperadamente?

¿Qué estrategias puedes aplicar para optimizar el rendimiento de una consulta con varias subqueries anidadas?

¿En qué escenarios productivos usarías subqueries en la cláusula FROM en lugar de crear vistas temporales?

¿Cuál es el impacto de un mal uso de subqueries en la integridad del análisis y la toma de decisiones en un contexto organizacional?

Referencias bibliográficas

- Beaulieu, A. (2009). Learning SQL (2nd ed.). O'Reilly Media.
- Melton, J., & Simon, A. R. (2002). SQL: 1999 – Understanding Relational Language Components. Morgan Kaufmann.
- McKinney, W. (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.